

sh2elf: Compiling POSIX Shell Scripts to Native x86-64 ELF Binaries

Ivan Gaydardzhiev
Independent Research

This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Abstract. sh2elf is a single-pass compiler that translates POSIX shell scripts into statically linked ELF64 executables for Linux on x86-64. It handles the full shell language subset including pipelines, control flow, parameter expansion, arithmetic, signal handling, and a large set of built-in commands, all without invoking an external shell at runtime. The compiler is written in C, totals roughly 3100 lines, and produces binaries that execute 11.28 times faster than the equivalent bash script on a comprehensive benchmark exercising the entire supported language. The output binary carries no shared library dependencies and communicates with the kernel exclusively through raw Linux system calls.

INTRODUCTION

The POSIX shell is not a fast language. Every invocation parses text, forks subprocesses to evaluate command substitutions, and interprets variable expansions at runtime. For scripts that run on hot paths or that are shipped as part of a distribution's init or package infrastructure, this overhead is constant and unavoidable. Attempts to speed up shell execution have historically fallen into two categories: faster interpreters such as dash, and transpilation to another interpreted language such as Python or Perl. Neither approach eliminates the interpreter from the execution path.

sh2elf takes a different position. Given a shell script, it compiles it once and produces a native ELF64 binary that the Linux kernel can execute directly. The resulting program contains no interpreter, no shared libraries, and no calls back into the shell. All control flow is native branches. All I/O goes through inline system call sequences. Variable state lives in a BSS hashtable. Pipelines are implemented with `fork`, `pipe`, and `dup2` at the call site, emitted as machine code.

The practical consequence is a binary that runs an order of magnitude faster than bash on identical workloads and that can be copied to any x86-64 Linux system and executed without any runtime dependencies.

COMPILER ARCHITECTURE

The compiler is structured as a front end, an intermediate representation, a code generator, and an ELF writer. All four stages are contained in `sh2elf.c`, a single C source file. There are no external parser generators, no LLVM passes, and no linking to a C runtime.

Front end and IR

The front end reads a shell script and produces a flat array of IR statements. Each IR statement holds a condition field indicating whether it participates in a `&&` or `||` chain, and a pipeline field that lists the stages of the pipeline as argument vectors. This is a three-address representation in spirit: the front end resolves variable references and quoting rules into slot indices rather than keeping raw text around, though for dynamic values the full resolution is deferred to runtime code sequences.

The IR layer decouples parsing from code generation cleanly enough that the compiler can also emit a textual dump of the IR for inspection, and can produce a source-annotated disassembly listing that maps each shell line to the x86-64 instructions it generated.

Code generator

The code generator walks the IR and emits x86-64 machine code directly into a growable byte buffer. There is no assembler text involved at any stage. Each instruction is written byte by byte using small helper functions that encode register-immediate and register-register forms.

For built-in commands, the generator emits inline system call sequences. A `pwd` statement becomes a `mov rdi / mov rsi / mov rax, 79 / syscall` sequence. A `printf` with a known string argument becomes a `write syscall` with the string address patched in from the string pool. For external commands, the generator emits a `fork/execve` pair at the call site. The compiler selects the full path from a fixed list of prefixes at compile time, so the runtime does not search `PATH`.

Arithmetic expansion is handled by a 64-bit recursive descent integer evaluator that emits x86-64 arithmetic in-

structions directly. The expression `(( A * B + C ))` compiles to a small sequence of `mov`, `imul`, and `add` instructions operating on `rax`.

String pool

All string literals are collected into a `StrPool` at compile time. The pool uses linear search for deduplication: before adding a string it scans for an existing match, and duplicate strings are not emitted. The pool is placed in the binary in a read-only segment, and the code generator emits relocations that the ELF writer resolves before writing the file.

ELF writer

The ELF writer constructs the output binary entirely in memory and writes it in a single call. It builds an ELF64 header with two program headers: one `PT_LOAD` segment covering the text and read-only data at virtual address `0x400000` with read and execute permissions, and one `PT_LOAD` segment covering the BSS at `0x600000` with read and write permissions. The entry point is set to the first generated instruction, which immediately follows the ELF and program headers in the file.

Before writing, the ELF writer runs the peephole optimizer on the code buffer. The optimizer replaces `mov rax, 0` with `xor eax, eax` and removes unconditional jumps that target the following instruction. It then patches all pending relocations, filling in the absolute virtual addresses for string pool entries that the code generator left as placeholders.

MEMORY MODEL

The generated binary uses three memory regions. The text segment holds executable code beginning at `0x400000`. Immediately following the code in the same segment is the read-only string pool. The BSS segment begins at `0x600000` and holds all mutable state.

The BSS layout is fixed across all generated binaries. Offset 0 holds the exit status register, an 8-byte slot that every command updates with its return code. Offset 8 holds the positional parameter frame: slots for `$0` through `$9`, plus `$#`, `$@`, and `$*`. These are populated by the binary prologue, which reads `argc` and `argv` from the Linux initial stack before executing any shell statements.

Offset 256 begins the variable hashtable, which uses open addressing with linear probing. Each slot holds a key pointer and a value pointer. Variable lookup and assignment are emitted as inline code sequences that hash the key string, probe the table, and read or write the value pointer without any function call overhead, since the table address is a compile-time constant.

Offset 4352 is the I/O scratch buffer, a 64 KB region used by streaming built-in commands for their read and write syscall loops.

PIPELINE IMPLEMENTATION

Pipelines are compiled into a sequence of `pipe`, `fork`, `dup2`, `close`, and `wait4` system calls emitted inline at the point where the pipeline appears in the source. For a two-stage pipeline `A | B`, the generated code calls `sys_pipe` to create a file descriptor pair, forks a child for `A` that redirects its stdout to the write end of the pipe, and forks a child for `B` that redirects its stdin to the read end. The parent closes both ends and waits for both children. The exit status of the pipeline is taken from the last stage and stored into the BSS status register.

Pipelines of arbitrary length are supported. Each additional stage introduces one more `fork/dup2/close` block, and the parent accumulates `wait4` calls for all children, discarding all statuses except the last.

`SIGPIPE` handling is set up by a `sys_rt_sigaction` call in the binary prologue with `SIG_DFL`, restoring default signal disposition for pipeline stages that may receive a broken pipe.

VARIABLE HANDLING

Variables are dynamic at runtime. The compiler cannot statically determine all variable names or their lifetimes in the general case because of `eval` and because variables may be set by external commands through environment inheritance. The BSS hashtable accommodates this: any variable name encountered at runtime is hashed and stored as a key-value pair of `char*` pointers.

For variables whose names and values are known at compile time, the compiler can sometimes emit the value directly as a string pool reference and bypass the hashtable entirely. This applies to simple cases where no variable is involved, and to arithmetic expressions where both operands are numeric literals.

Parameter expansions are emitted as inline runtime routines. The expansion `${VAR##pat}` compiles to a sequence that scans the value of `VAR` from left to right evaluating `fnmatch` at each offset, takes the longest prefix that matches `pat`, and returns the remaining suffix. The expansion `${VAR/pat/rep}` performs substitution rather than stripping by the same mechanism. All these operations work directly on string data in the BSS scratch buffer.

CONTROL FLOW

Control flow constructs compile to native branch sequences with forward or backward jumps patched after the target address is known.

An `if` statement compiles to: evaluate the condition pipeline, load the status from the BSS status register, compare against zero, and branch forward over the then-body if non-zero. The `else` branch, if present, is prefixed by an unconditional jump over it from the end of the then-body.

Patch addresses are recorded on a compile-time stack and filled in once the target offset is known.

`while` and `until` loops record the loop top address before emitting the condition, emit the condition, branch past the loop body on failure (or success for `until`), emit the body, and emit an unconditional backward jump to the loop top. `break N` and `continue N` are implemented with a backpatching stack that records patch sites for each nesting level.

`case` statements compile to a chain of `fnmatch` evaluations. Each pattern compiles to a test-and-branch sequence. On a match the compiler emits the corresponding body followed by an unconditional jump to the end of the entire `case` block.

BUILT-IN COMMANDS

sh2elf implements a large set of shell built-in commands as inline code rather than external binaries, using raw system calls without any C standard library involvement. This is the second main source of the compiler’s performance advantage: where `bash` forks a subprocess to run `grep` or `sort`, sh2elf emits an inline loop of `read/write` syscalls that processes the stream without forking.

The streaming built-ins, including `grep`, `sort`, `uniq`, `cut`, `tr`, `head`, `tail`, `sed`, `awk`, `find`, `ls`, `ps`, `tar`, `gzip`, and `gunzip`, read from `stdin` through the BSS I/O buffer and write to `stdout` through the same buffer. The 64 KB buffer size matches typical kernel pipe buffer sizes and keeps memory pressure low.

File-manipulation built-ins such as `cp`, `mv`, `rm`, `mkdir`, `rmdir`, `touch`, `chmod`, `chown`, and `chgrp` are compiled to the corresponding syscalls directly. Process-related built-ins including `kill`, `killall`, `pgrep`, `pkill`, `ps`, and `nice` operate by scanning `/proc` through `sys_getdents64` syscalls, matching process names against the argument, and issuing `sys_kill` on matching PIDs.

The `eval` built-in is implemented as a runtime compilation step. The argument string is assembled, written to a temporary file, and the compiler is reinvoked on that file. This makes `eval` correct but not fast; a script that calls it in a hot loop will not benefit from compilation.

TOOLING

The compiler binary provides several operating modes beyond basic compilation. The `--list` mode emits a source-annotated listing that pairs each shell source line with the x86-64 instructions it generated, useful for verifying code generation correctness. The `--dump-ir` mode prints the three-address IR before code generation, showing each statement’s condition flag and pipeline argument vectors. The `--emit-obj` mode produces a relocatable ELF .o

object file that can be linked into a C program, allowing shell logic compiled by sh2elf to be called from C.

A language server, `sh2elf-lsp`, implements a subset of the Language Server Protocol and provides real-time parse error reporting with line and column positions, and hover information for built-in commands.

A differential test harness, `test_diff`, compiles each test script with sh2elf and runs both the compiled binary and `bash` on the same input, comparing `stdout` and exit status byte-for-byte. The test suite covers 89 integration tests across the full language subset, all of which pass. A grammar-based fuzzer, `fuzz_sh`, generates random but syntactically valid shell scripts and runs them through the compiler and code generator; 100 iterations completed without triggering a crash or assertion failure.

PERFORMANCE

The benchmark runs a 130-line shell script that exercises every supported language construct: variables, arithmetic, control flow, pipelines, parameter expansions, string operations, file manipulation, and built-in commands. The script is run 50 times for each executor.

Executor	Total (50 runs)	Per run	Speedup
bash	3.668 s	73.3 ms	1.0×
sh2elf	0.325 s	6.5 ms	11.28×

Table 1: Execution time on a comprehensive POSIX benchmark.

The speedup comes from two sources. First, sh2elf eliminates interpreter startup and text parsing on every run. Second, built-in commands that `bash` implements by forking an external process are emitted as inline syscall loops in the sh2elf binary, removing the `fork/exec/wait` overhead for each invocation.

FORMAL SEMANTICS

The compilation function $\mathcal{C}$ maps a shell script to an ELF64 binary represented as a tuple $\langle \text{Code}, \text{StrPool}, \text{Relocations}, \text{BSS} \rangle$.

For simple built-in commands with statically known arguments, compilation is fully determined at compile time. The command `printf s` compiles to the sequence:

```

 $\mathcal{C}[\text{printf } s] = \text{mov rdi}, 1
                  \text{mov rsi}, \text{Addr}(s)
                  \text{mov rdx}, \text{Len}(s)
                  \text{mov rax}, 1
                  \text{syscall}$ 
```

A two-stage pipeline $\mathcal{C}_1 \mid \mathcal{C}_2$ compiles to a fixed sequence: `sys_pipe` creates the descriptor pair; a first

`sys_fork` produces a child that calls `sys_dup2(fd1, 1)`, closes *fd*₀, and executes *C*₁; a second `sys_fork` produces a child that calls `sys_dup2(fd0, 0)`, closes *fd*₁, and executes *C*₂; the parent closes both descriptors, issues `sys_wait4` for both children, and stores the status of *C*₂ into `BSS[0]`.

Short-circuit operators compile to conditional branches on the `BSS` status register. The `&&` operator compiles as follows: emit `C[P1]`, then `cmp [BSS0], 0`, then `jne Lskip`, then `C[P2]`, then place `Lskip`. The `||` operator is symmetric with `jne` in place of `jne`.

Parameter stripping expansions of the form `${VAR##pat}` evaluate `fnmatch(pat, S[0..k])` for $k = 1, \dots, |S|$ and return the suffix after the longest matching prefix. Suffix stripping `${VAR%%pat}` mirrors this from the right. Dynamic substitution `${VAR//pat/rep}` scans the buffer and replaces every non-overlapping match of `pat` with `rep`.

LIMITATIONS

sh2elf targets a well-defined subset of POSIX shell. Constructs outside that subset are rejected with a parse error at compile time rather than silently miscompiled. The source language deliberately excludes features that cannot be compiled efficiently or that require a runtime interpreter: dynamic `source` of unknown files, co-processes, job control signals, and interactive features such as readline history are not supported.

The `eval` built-in is an exception. Because its argument is an arbitrary string assembled at runtime, it requires a

runtime compilation step, making it correct but not fast.

The output binary carries no dynamic relocation information. The fixed virtual addresses are suitable for statically linked executables where no dynamic linker is involved, which is exactly the use case sh2elf targets.

CONCLUSION

sh2elf demonstrates that the semantic gap between POSIX shell and native machine code is smaller than it might appear. The core shell execution model, pipelines, redirections, variable expansion, and control flow, maps directly to sequences of Linux system calls and x86-64 branch instructions. A compiler that performs this translation statically removes the interpreter overhead entirely and produces binaries that run at speeds appropriate for systems software rather than scripting.

The compiler is self-contained, requiring no external tools beyond a C compiler to build. All built-in command implementations use raw system calls. The resulting binaries have no shared library dependencies and no runtime requirements beyond the Linux kernel ABI. For shell scripts that need to be fast, distributable as single files, or embedded in environments where a shell may not be present, sh2elf provides a practical compilation path.

The source code for sh2elf is available under the GNU General Public License version 3. Copyright © 2025–2026 Ivan Gaydardzhiev. This paper is licensed under CC BY 4.0.